

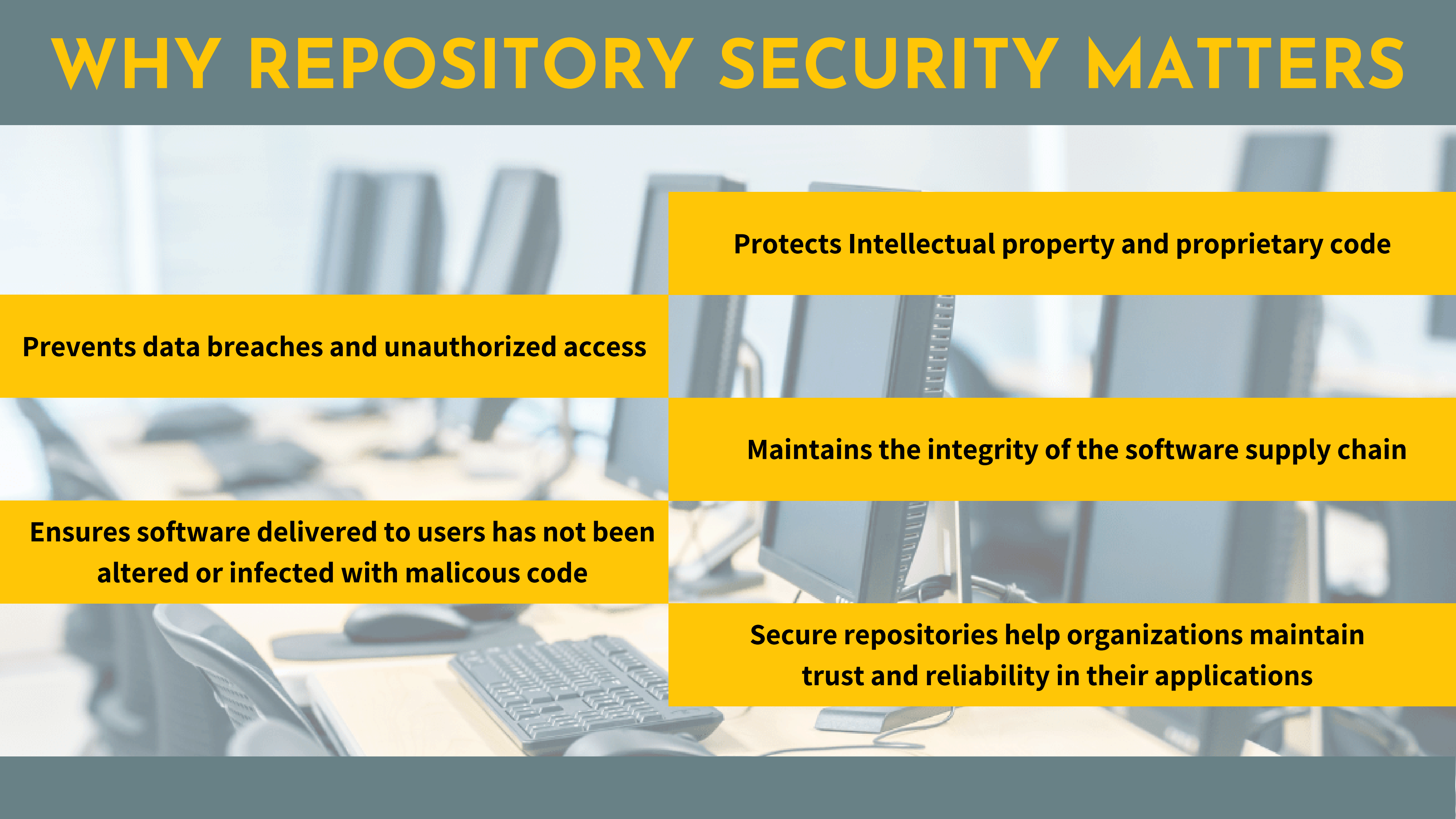
SECURITY CONTROLS IN SHARED SOURCE CODE REPOSITORIES

Alexis Mitchell

CSD380

March 8, 2026

WHY REPOSITORY SECURITY MATTERS



Protects Intellectual property and proprietary code

Prevents data breaches and unauthorized access

Maintains the integrity of the software supply chain

Ensures software delivered to users has not been altered or infected with malicious code

Secure repositories help organizations maintain trust and reliability in their applications

COMMON SECURITY THREATS



Unauthorized access

Attackers gain access to
repositories or
developer accounts

Passwords, API keys, or
tokens accidentally
committed into code



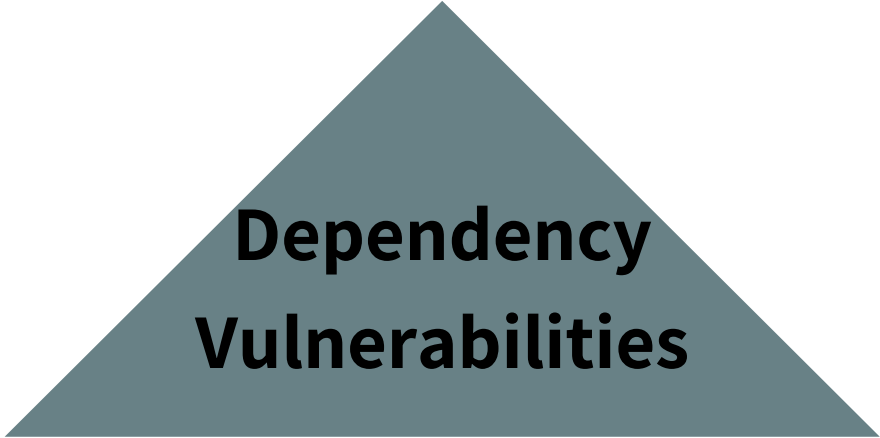
Exposed Secrets



Malware Injection

Malicious code inputed
into legitimate
applications

Third-party libraries
containing security
flaws



Dependency Vulnerabilities

ACCESS CONTROL AND AUTHENTICATION

Role-Based Access Control

- Limits repository access based on user roles
- Developers only receive permissions they need

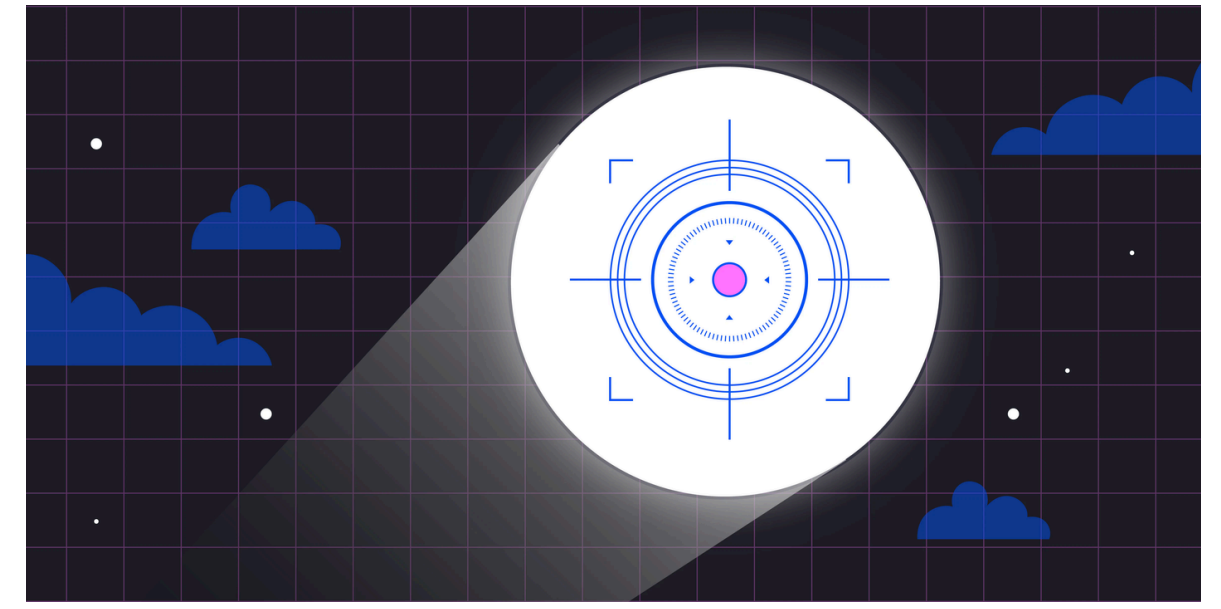
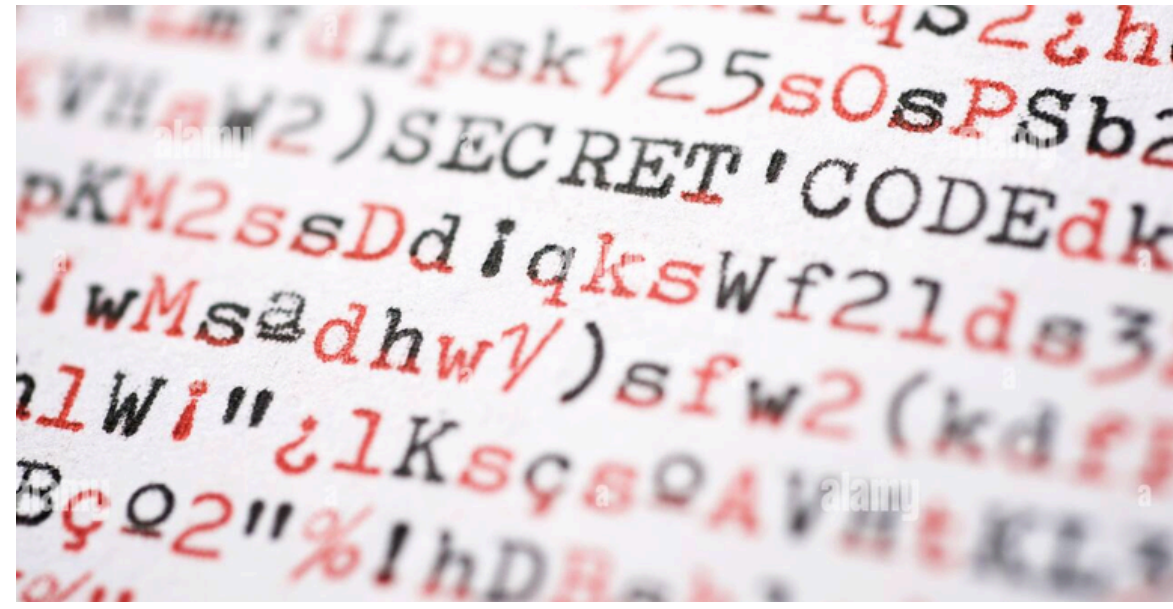
Multi-Factor Authentication (MFA)

- Require more than a password to log in
- Implement verification codes or biometrics

Credential Protection

- Avoid sharing passwords or private keys
- Rotate access keys regularly and revoke access immediately when its no longer needed

PROTECTING SENSITIVE INFORMATION



Secret Management

- Avoid hardcoding credentials directly in source code
- Use secure secret management tools such as:
 - HashiCorp Vault
 - Built-in repository secret managers

Separate Secrets from Code

- Prevents exposure if the repository becomes public or compromised

Automated Secret Scanning

- Helps detect leaked credentials before they can be abused



CODE INTEGRITY AND QUALITY CONTROLS



Code Reviews

- Required before merging changes
 - Helps catch vulnerabilities, logical errors, and design flaws early
-

Branch Protection

- Prevents direct changes to critical branches
 - Requires approvals and automated checks before merging
-

Code Signing

- Ensures code changes are verified and made by trusted contributors
-

DEPENDENCY AND SUPPLY CHAIN SECURITY

Pinned Dependencies

Specify exact library versions to prevent malicious updates

Maintained Dependencies

Use actively maintained libraries that receive security patches

Software Bill of Materials (SBOM)

Tracks all components used in a project to improve transparency

Signed Releases

Confirms that software packages come from trusted maintainers

MONITORING AND TESTING



Repository Activity Monitoring

Alerts for suspicious activity or unauthorized access attempts



Fuzz Testing

Sends unexpected inputs to applications to find hidden vulnerabilities



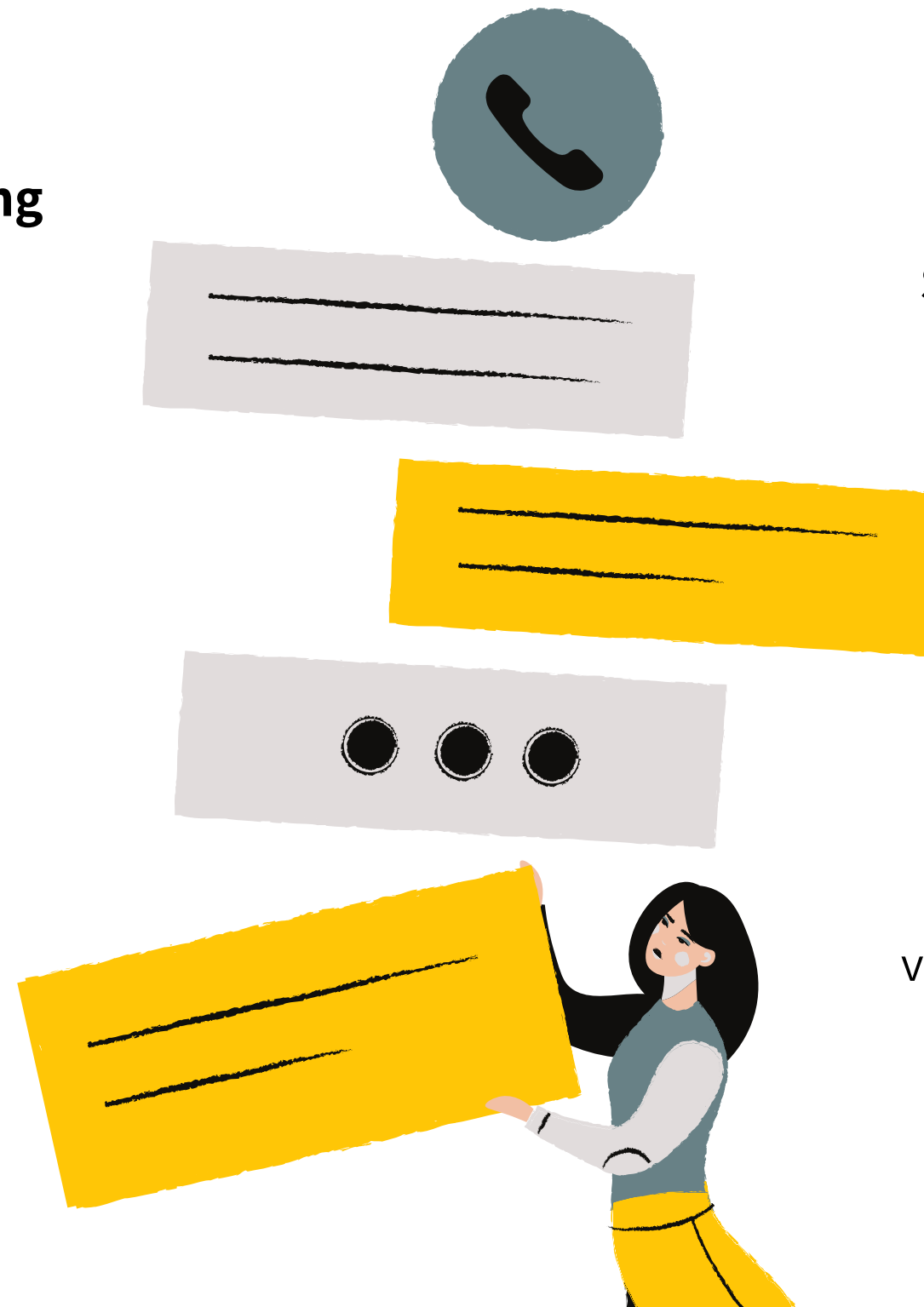
Vulnerability Scanning

Use tools like:

- Static Application Security Testing (SAST)
- Software Composition Analysis (SCA)

CI Pipeline Testing

Auto checks for errors or vulnerabilities before code is merged



References



- **Harness. (2026, February 11). Best practices for securing code repositories. Harness.io.**
<https://www.harness.io/harness-devops-academy/secure-code-repositories-best-practices>
- **NCSC. (2019, February 20). Protect your code repository. National Cyber Security Centre - NCSC.GOV.UK.**
<https://www.ncsc.gov.uk/collection/developers-collection/principles/protect-your-code-repository>
- **Rose, J. (2025, July 24). 12 best practices for Secure Code Repositories (2025 guide). Checkmarx.**
<https://checkmarx.com/supply-chain-security/repository-health-monitoring-part-2-essential-practices-for-secure-repositories/>